

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ІВАНА ФРАНКА
Факультет електроніки та комп'ютерних технологій

З ЛАБОРАТОРНОЇ РОБОТИ №3
з дисципліни "Об'єктно орієнтоване програмування (C++)"
на тему "Створення власного класу. Методи розв'язку рівнянь"

Виконала:

студентка 1 курсу
групи ФЕП-12
Ступницька Юля Ярославівна

Перевірив:

асистент
Левуш Павло Назарович

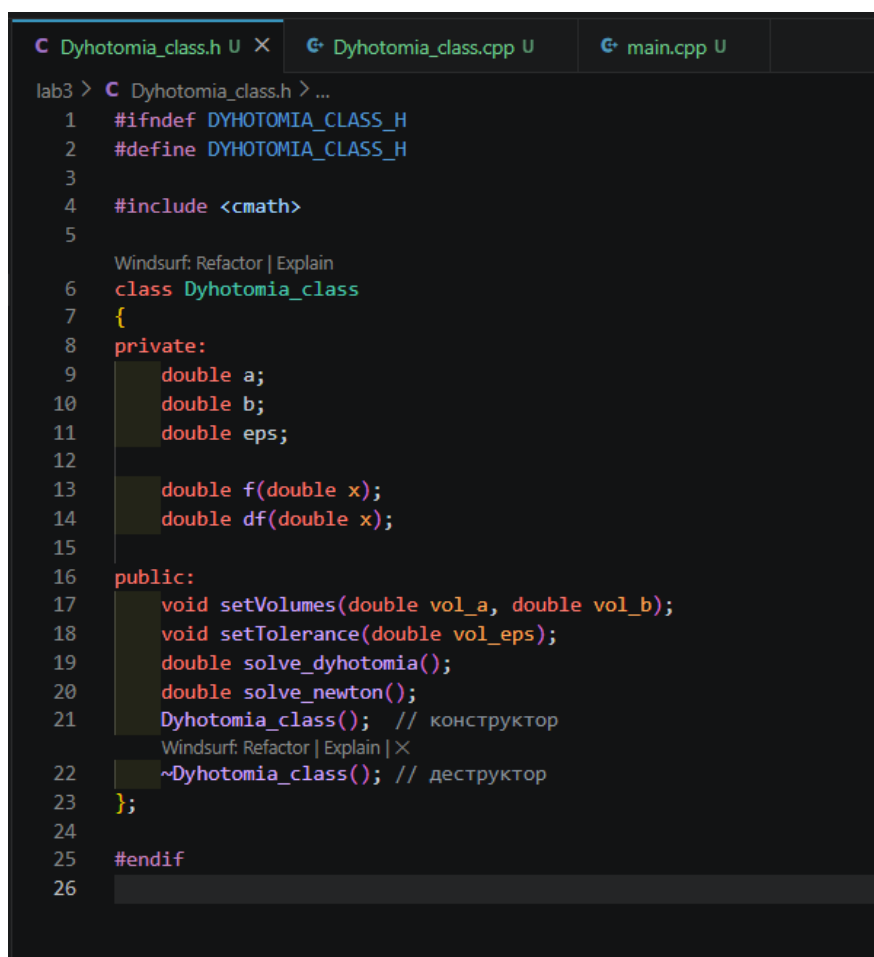
Мета роботи

Засвоїти структуру опису класу в мові C++, навчитись реалізовувати функціонал та використовувати класи в основній програмі. Створити власні класи з реалізаціями розв'язання нелінійного рівняння методами дихотомії та дотичних з використанням принципів інкапсуляції.

Хід роботи

Крок 1. Створення заголовкового файлу (Dyhotomia_class.h)

Описано клас Dyhotomia_class з приватними полями та публічними методами. Використано принцип інкапсуляції — дані приховані в секції private, доступ до них реалізований через public методи.



```
lab3 > C Dyhotomia_class.h > ...
1  #ifndef DYHOTOMIA_CLASS_H
2  #define DYHOTOMIA_CLASS_H
3
4  #include <cmath>
5
6  Windsurf: Refactor | Explain
7  class Dyhotomia_class
8  {
9  private:
10     double a;
11     double b;
12     double eps;
13
14     double f(double x);
15     double df(double x);
16
17 public:
18     void setVolumes(double vol_a, double vol_b);
19     void setTolerance(double vol_eps);
20     double solve_dyhotomia();
21     double solve_newton();
22     Dyhotomia_class(); // конструктор
23     ~Dyhotomia_class(); // деструктор
24 };
25 #endif
26
```

Крок 2. Реалізація методів класу (Dyhotomia_class.cpp)

Реалізовано всі методи класу:

- конструктор та деструктор
- setVolumes() та setTolerance() — для задання параметрів

- $f(x)$ — рівняння варіанту 7: $f(x) = x - 2 + \sin(1/x)$
- $df(x)$ — похідна: $f'(x) = 1 - \cos(1/x) / x^2$
- `solve_dyhotomia()` — метод дихотомії
- `solve_newton()` — метод Ньютона (дотичних)

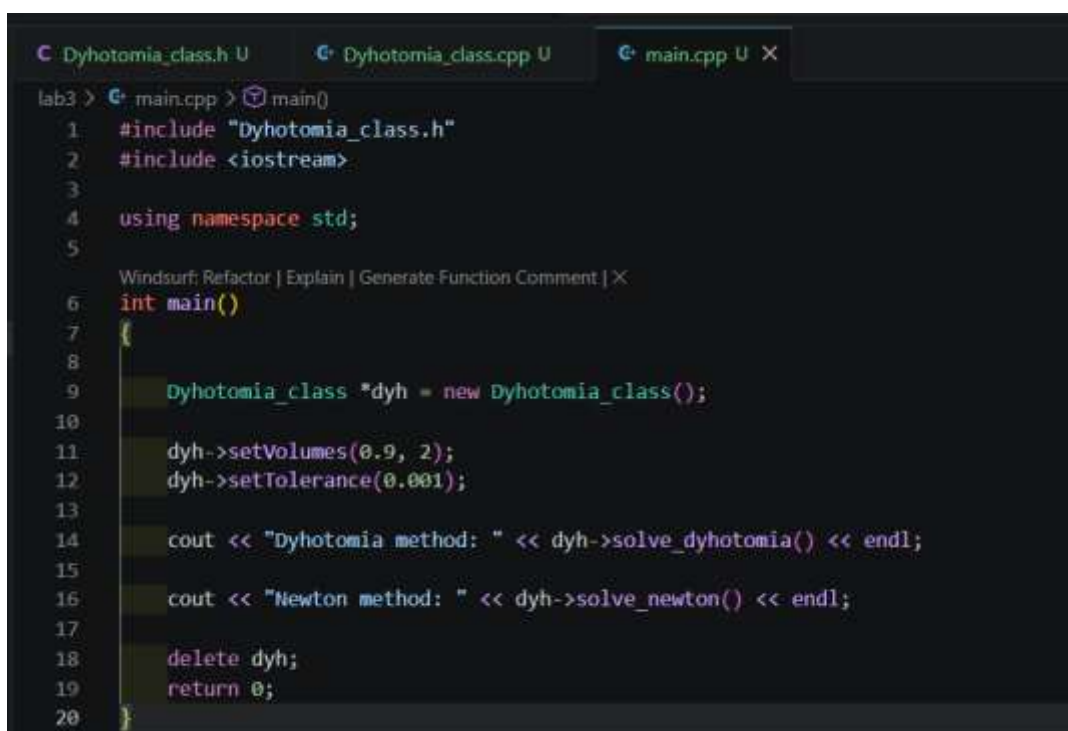
```

Dyhotomia_class.h | Dyhotomia_class.cpp | main.cpp
lab3 > Dyhotomia_class.cpp > solve_newton()
1  #include "Dyhotomia_class.h"
2
3  Dyhotomia_class::Dyhotomia_class()
4  {
5      a = 0;
6      b = 0;
7      eps = 0.001;
8  }
9
10 Dyhotomia_class::~Dyhotomia_class()
11 {
12 }
13
14 void Dyhotomia_class::setVolumes(double vol_a, double vol_b)
15 {
16     a = vol_a;
17     b = vol_b;
18 }
19
20 void Dyhotomia_class::setTolerance(double vol_eps)
21 {
22     eps = vol_eps;
23 }
24
25 double Dyhotomia_class::f(double x)
26 {
27     return x - 2 + sin(1/x);
28 }
29
30 double Dyhotomia_class::df(double x)
31 {
32     return 1 - cos(1/x) / (x * x);
33 }
34
35 double Dyhotomia_class::solve_dyhotomia()
36 {
37     double c;
38     while (b - a > eps)
39     {
40         c = (a + b) / 2;
41         if (f(c) * f(a) < 0)
42             b = c;
43         else
44             a = c;
45     }
46     return (a + b) / 2;
47 }
48
49 double Dyhotomia_class::solve_newton()
50 {
51     double x = (a + b) / 2;
52     double x_old;
53     do
54     {
55         x_old = x;
56         x = x - f(x) / df(x);
57     } while (fabs(x - x_old) > eps);
58     return x;
59 }
60
61

```

Крок 3. Основна програма (main.cpp)

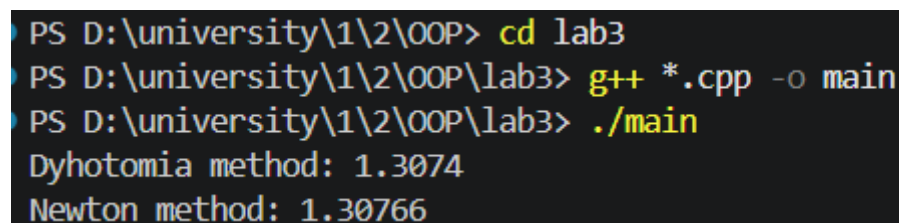
Реалізовано створення об'єкту класу через оператор new, задання параметрів та виклик обох методів розв'язку.



```
lab3 > C: main.cpp > main()
1  #include "Dyhotomia_class.h"
2  #include <iostream>
3
4  using namespace std;
5
6  Windsurf: Refactor | Explain | Generate Function Comment | X
7  int main()
8  {
9      Dyhotomia_class *dyh = new Dyhotomia_class();
10
11     dyh->setVolumes(0.9, 2);
12     dyh->setTolerance(0.001);
13
14     cout << "Dyhotomia method: " << dyh->solve_dyhotomia() << endl;
15
16     cout << "Newton method: " << dyh->solve_newton() << endl;
17
18     delete dyh;
19     return 0;
20 }
```

Крок 4. Результат виконання програми

Програму скомпільовано та запущено. Обидва методи дали близькі результати для кореня рівняння $f(x) = x - 2 + \sin(1/x)$ на проміжку $[0.9; 2]$:



```
PS D:\university\1\2\OOP> cd lab3
PS D:\university\1\2\OOP\lab3> g++ *.cpp -o main
PS D:\university\1\2\OOP\lab3> ./main
Dyhotomia method: 1.3074
Newton method: 1.30766
```

Крок 5. GitHub

Проект опубліковано на GitHub.

<https://github.com/STUPNITSKA/lab>

Висновок

У роботі було реалізовано власний клас Dyhotomia_class з використанням принципу інкапсуляції. Реалізовано два методи розв'язання нелінійного рівняння — метод дихотомії та метод Ньютона

(дотичних). Освоєно роботу з класами в C++, конструкторами, деструкторами, модифікаторами доступу public та private, а також оператором new та delete для динамічного створення об'єктів.